

Simulation-Based Inference

Likelihood-based Bayesian Inference

Paul Bürkner

Markov Chain Monte Carlo (MCMC) Sampling

A Markov Chain is a sequence of values where the value at position t is based only on the former value at position $t - 1$:

$$\theta^{(1)} \rightarrow \theta^{(2)} \rightarrow \theta^{(3)} \rightarrow \dots \rightarrow \theta^{(S)}$$

Probabilistic formulation:

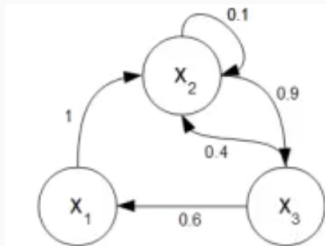
$$p(\theta^{(t)} \mid \theta^{(t-1)}, \theta^{(t-2)}, \dots, \theta^{(1)}) = p(\theta^{(t)} \mid \theta^{(t-1)})$$

MCMC sampling:

- Start with a random initial value from $p(\theta^{(1)})$
- Sample from $p(\theta^{(t)} \mid \theta^{(t-1)})$ for increasing t

Example: Markov chains with three states

Consider the transition probabilities:



0	1	0
0	0.1	0.9
0.6	0.4	0

The transition matrix (right-hand side above) is $Q := p(\theta^{(t+1)} \mid \theta^{(t)})$

Task: Find the *stationary* distribution $p(\theta)$ that satisfies

$$p(\theta)Q = p(\theta)$$

Source: <https://towardsdatascience.com/monte-carlo-markov-chain-mcmc-explained-94e3a6c8de11>

Example: Markov chains with three states

Start somewhere, say, at $p(\theta^{(1)}) = (0.5, 0.2, 0.3)$

Move along the Markov Chain according to Q

- $p(\theta^{(2)}) = p(\theta^{(1)})Q = (0.18, 0.64, 0.18)$
- $p(\theta^{(3)}) = p(\theta^{(2)})Q = (0.108, 0.316, 0.576)$
- $p(\theta^{(4)}) = p(\theta^{(3)})Q = (0.346, 0.370, 0.284)$
- ...
- $p(\theta^{(49)}) = p(\theta^{(48)})Q = (0.221, 0.410, 0.369)$
- $p(\theta^{(50)}) = p(\theta^{(49)})Q = (0.221, 0.410, 0.369)$

Example: Markov chain with three states (sampling version)

Choose a total number of S draws, say $S = 1000$

Start somewhere, say at $p(\theta^{(1)}) = (0.5, 0.2, 0.3)$

Sample one value at each time $t = 1, \dots, S$:

- $\theta^{(1)} \sim p(\theta^{(1)})$
- $\theta^{(2)} \sim p(\theta^{(2)} \mid \theta^{(1)}) = Q_{\theta^{(1)}}$
- $\theta^{(3)} \sim p(\theta^{(3)} \mid \theta^{(2)}) = Q_{\theta^{(2)}}$
- ...
- $\theta^{(1000)} \sim p(\theta^{(1000)} \mid \theta^{(999)}) = Q_{\theta^{(999)}}$

Store all $\theta^{(1:S)}$ in a vector

Throw away all $\theta^{(t)}$ with $t \leq t^*$, say $t^* = 50$

Use $\theta^{((t^*+1):S)}$ as draws from the stationary distribution $p(\theta)$

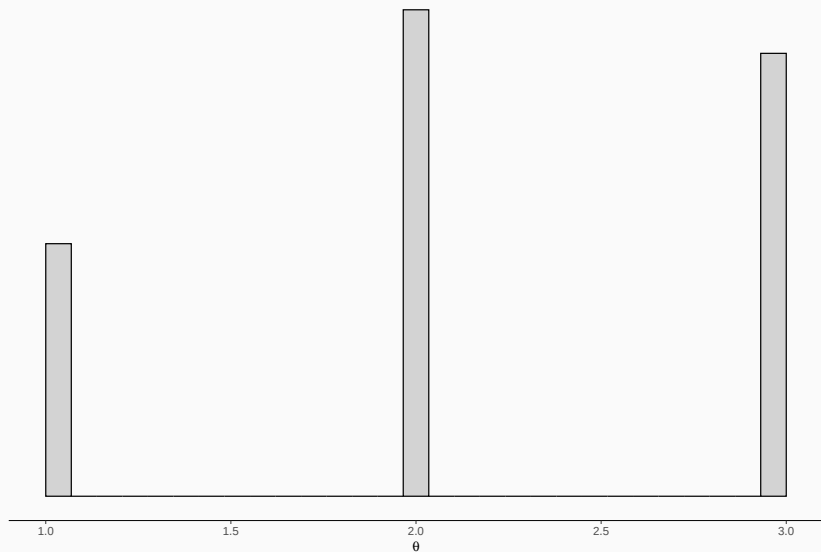
Example: Markov chain with three states (sampling version)

```
space = c(1, 2, 3)
p1 = c(0.5, 0.2, 0.3)
S = 1000
tstar = 50
```

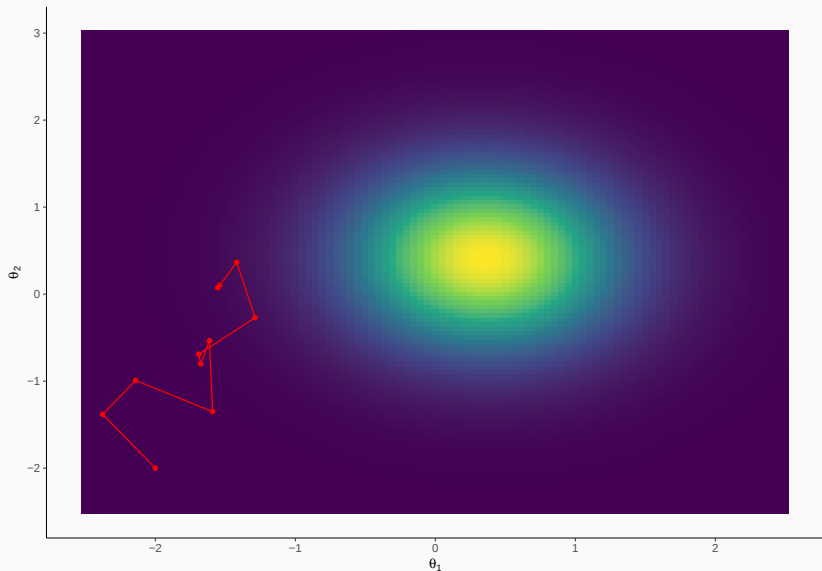
```
theta = rep(NA, S)
# initial value
theta[1] = sample(space, 1, prob = p1)
for (t in 2:S) {
  # Markov transitions
  theta[t] = sample(space, 1, prob = Q[theta[t-1], ])
}
# remove initial draws (burn-in / warmup period)
theta = theta[-(1:tstar)]
```

Example: Markov chain with three states (sampling version)

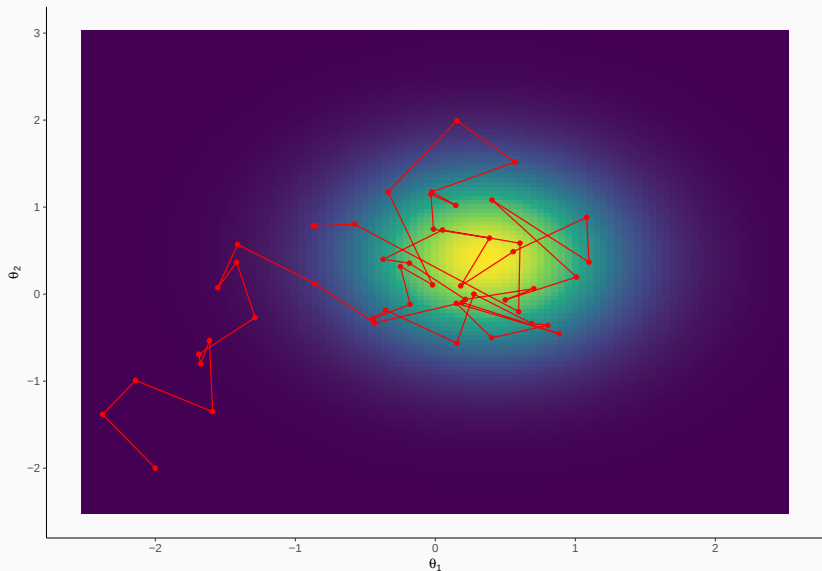
Results based on $\theta^{((t^*+1):S)} = \theta^{(51:1000)}$



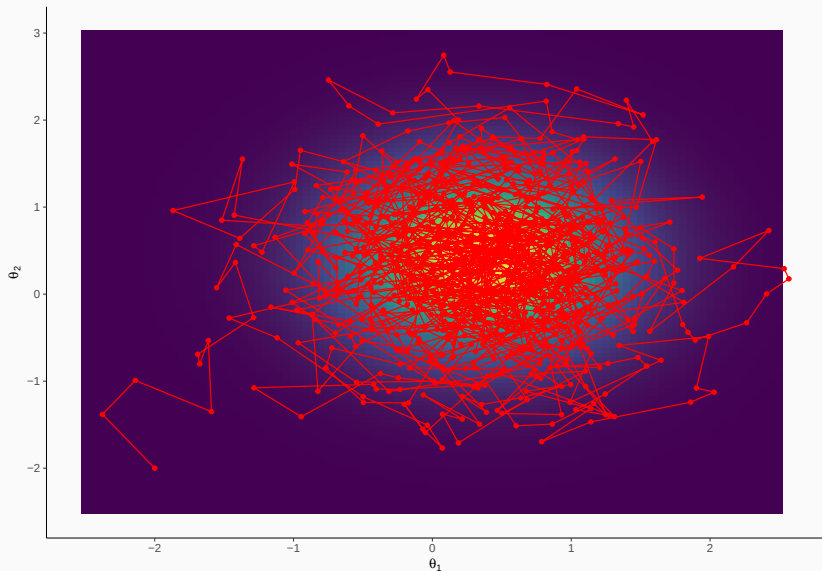
MCMC sampling in continuous spaces: 10 Iterations



MCMC sampling in continuous spaces: 50 Iterations



MCMC sampling in continuous spaces: 1000 Iterations



MCMC sampling for Bayesian inference

We try to draw MCMC samples in just the right way such that the stationary distribution of the Markov chain is equal to the posterior $p(\theta \mid y)$

We need:

- An initial value distribution $p(\theta^{(1)})$
- A transition distribution $p(\theta^{(t)} \mid \theta^{(t-1)}, y)$ such that $p(\theta^{(t)}) = p(\theta \mid y)$ for all t larger then some finite t^*
- Where t^* is the time of convergence to the stationary distribution
- Throw away the all values $\{\theta^{(t)} \mid t \leq t^*\}$ and keep the rest
- The kept values $\{\theta^{(t)} \mid t > t^*\}$ are draws from $p(\theta \mid y)$

Example: The Metropolis algorithm

(Start) Choose a total number of draws T and an initial value $\theta^{(1)}$.

Run the steps below for all $t = 1, \dots, T$:

- (1) Sample a possible new value $\theta^{(p)}$ based on a symmetric *proposal distribution* $g(\theta^{(p)} \mid \theta^{(t)})$
 - For example, use $N(\theta^{(t)}, \tau)$ as the proposal distribution, where τ serves as a tuning parameter controlling the *step-size*
- (2) Compute the ratio $\alpha = p(\theta^{(p)} \mid y) / p(\theta^{(t)} \mid y)$
- (3) Accept or reject the proposal:
 - If $\alpha \geq 1$, set $\theta^{(t+1)} = \theta^{(p)}$.
 - If $\alpha < 1$, set $\theta^{(t+1)} = \theta^{(p)}$ with probability α
 - Else, set $\theta^{(t+1)} = \theta^{(t)}$

(End) Throw away the first t^* draws as warmup.

Example: The Metropolis-Hastings algorithm

In case g is not symmetric, the ratio α becomes

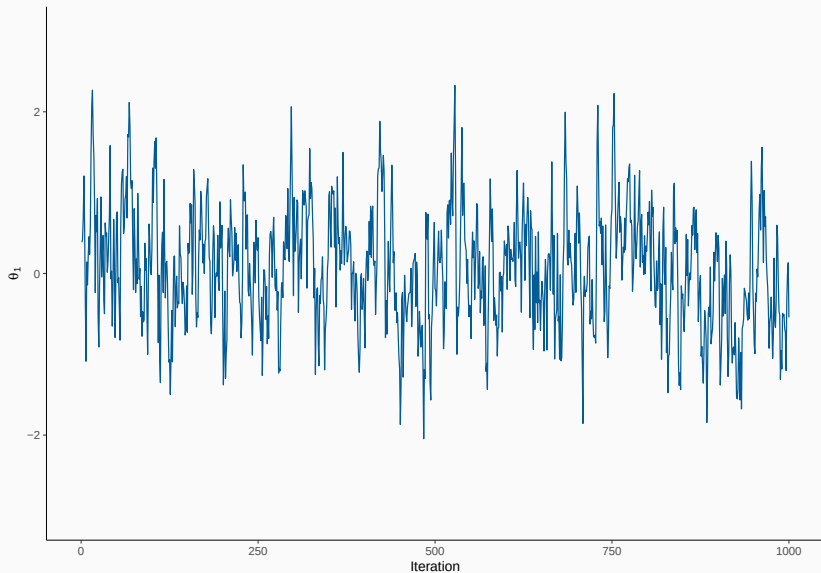
$$\alpha = \frac{p(\theta^{(p)} | y) g(\theta^{(t)} | \theta^{(p)})}{p(\theta^{(t)} | y) g(\theta^{(p)} | \theta^{(t)})}$$

- The rest of the algorithm remains the same
- This provides many more options for the proposal g
- g might be hard to choose in multi-dimensional settings

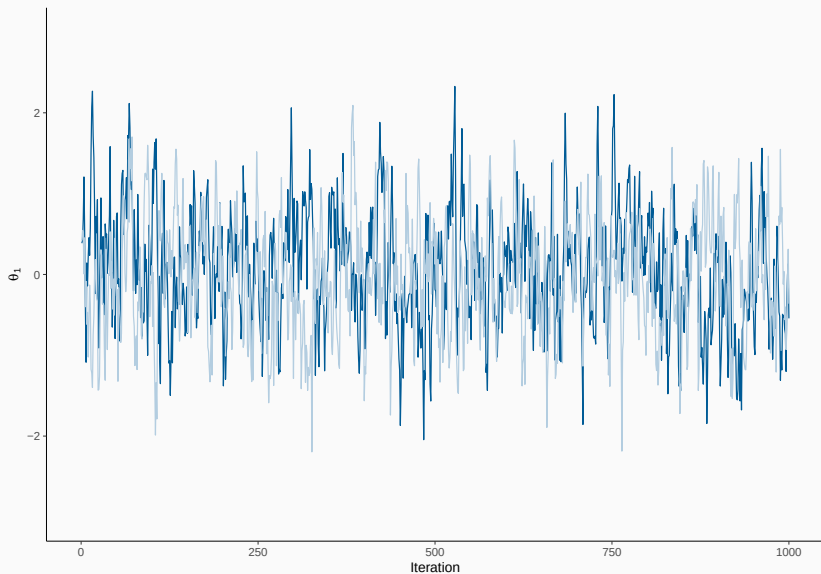
Metropolis-Hastings algorithm: interactive example

<https://chi-feng.github.io/mcmc-demo/app.html?algorithm=RandomWalkMH&target=banana>

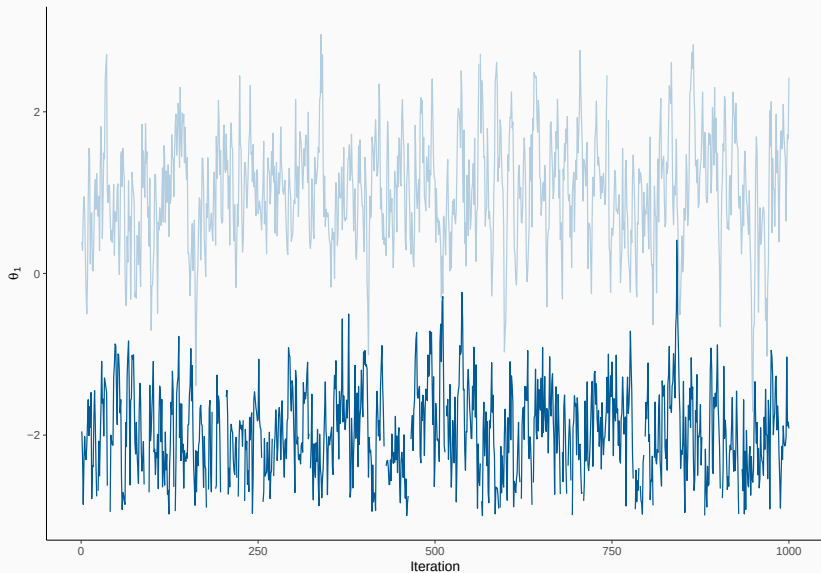
Trace Plots: Visualizing a Single Chain



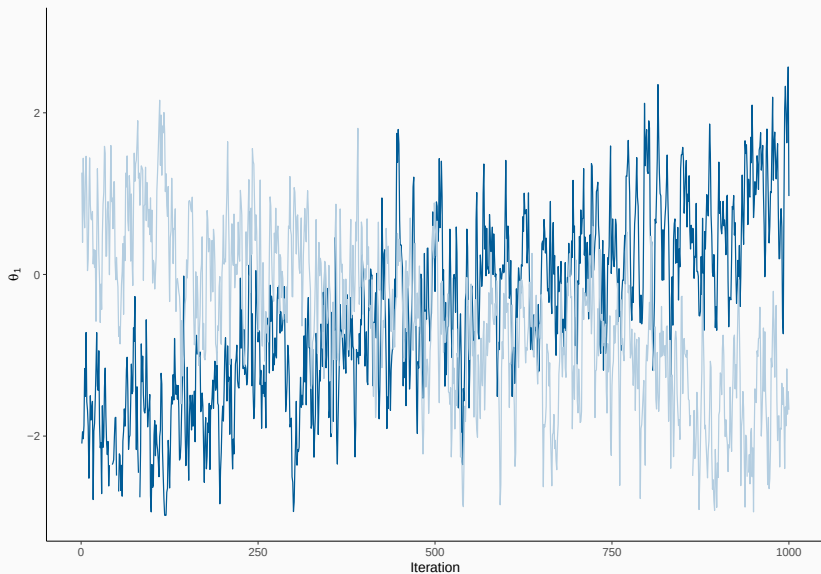
Trace Plots: Visualizing Multiple Chains



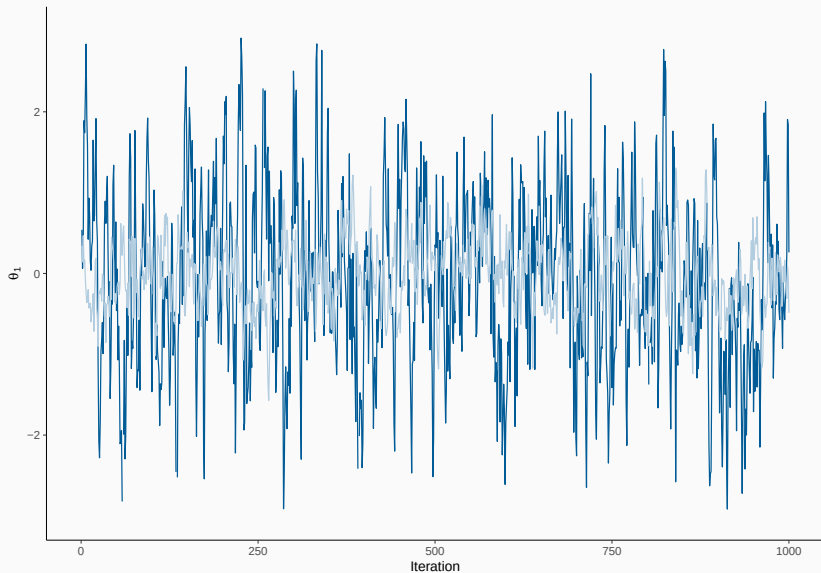
Chains with Different Locations



Non-Stationary Chains



Chains with Different Variances



Potential Scale Reduction Factor $\widehat{R}$

Suppose we have M chains each with N iterations such that $S = M N$

Between Chain Variance:

$$B = \frac{N}{M-1} \sum_{m=1}^M (\bar{\theta}^{(\cdot m)} - \bar{\theta}^{(\cdot)})^2$$

Within Chain Variance:

$$W = \frac{1}{M(N-1)} \sum_{m=1}^M \sum_{n=1}^N (\theta^{(nm)} - \bar{\theta}^{(\cdot m)})^2$$

Potential Scale Reduction Factor:

$$\widehat{R} = \sqrt{\frac{\frac{N-1}{N}W + \frac{1}{N}B}{W}}$$

Effective Sample Size (ESS)

The ESS measures how many independent draws contain the same amount of information as the S obtained dependent draws

$$\text{ESS} = \frac{S}{1 + 2 \sum_{t=1}^{\infty} \rho_t}$$

with lag- t autocorrelation

$$\rho_t = \rho_t(\theta) = \text{cor}(\theta^{(s)}, \theta^{(s+t)})$$

Estimation of $\sum_{t=1}^{\infty} \rho_t$ requires clever truncation

Assuming (*geometric*) *ergodicity* of a Markov Chain $\{\theta^{(s)}\}$:

$$\frac{1}{S} \sum_{s=1}^S f(\theta^{(s)}) \sim \text{Normal} \left(\mathbb{E}_{\theta|y}(f(\theta)), \sqrt{\frac{\text{Var}_{\theta|y}(f(\theta))}{\text{ESS}}} \right)$$

Gibbs sampling

Sample iteratively from the conditional posteriors

Start at some initial value $\theta^{(1)} = (\theta_1^{(1)}, \theta_2^{(1)}, \dots, \theta_K^{(1)})$

Sample:

- $\theta_1^{(2)} \sim p(\theta_1 \mid \theta_2^{(1)}, \dots, \theta_K^{(1)}, y)$
- $\theta_2^{(2)} \sim p(\theta_2 \mid \theta_1^{(2)}, \theta_3^{(1)}, \dots, \theta_K^{(1)}, y)$
- $\theta_3^{(2)} \sim p(\theta_3 \mid \theta_1^{(2)}, \theta_2^{(2)}, \theta_4^{(1)}, \dots, \theta_K^{(1)}, y)$
- ...
- $\theta_K^{(S)} \sim p(\theta_K \mid \theta_1^{(S)}, \dots, \theta_{K-1}^{(S)}, y)$

Challenges:

- We have to know how to sample from the conditional posteriors
- At each step, only one parameter is changed (rectangular steps)

Gibbs sampling: interactive example

<https://chi-feng.github.io/mcmc-demo/app.html?algorithm=GibbsSampling&target=banana>

Hamiltonian Monte Carlo

Special thanks to Michael Betancourt

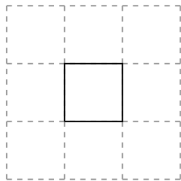
Source: Betancourt M. (2018). A Conceptual Introduction to Hamiltonian Monte Carlo. <https://arxiv.org/pdf/1701.02434.pdf>

Volume in high-dimensional spaces

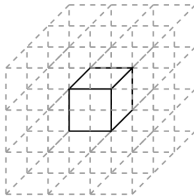
Volume around the mode diminishes in higher dimensions



(a)



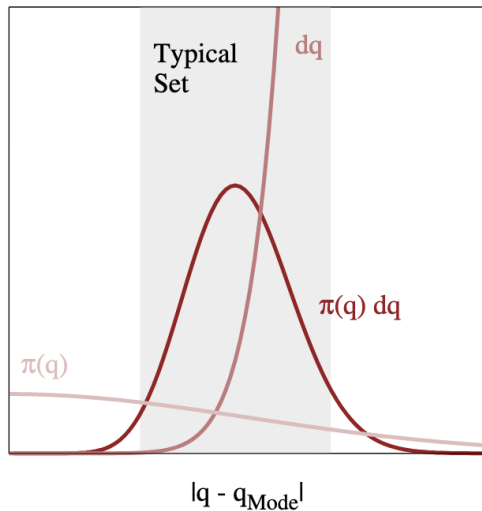
(b)



(c)

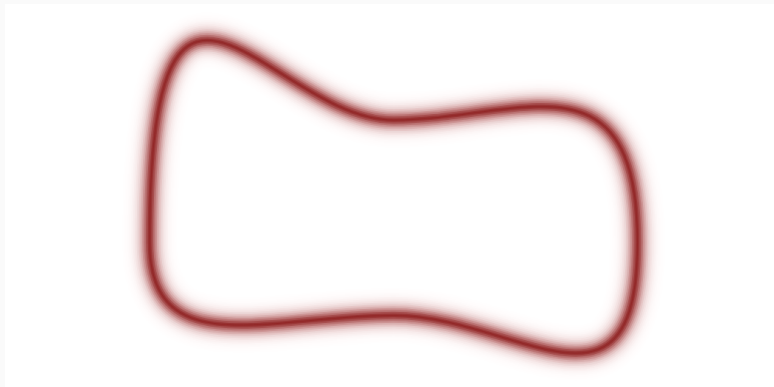
Volume vs. density

There is a sweet spot where the product of volume and density is high



The typical set

This sweet spot is called the typical set



The typical set: Metropolis exploration

Metropolis-Hasting algorithms have highly ineffective proposals in high dimensions



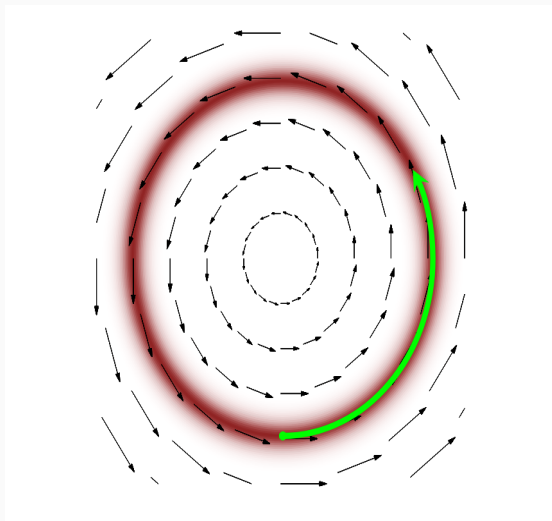
The typical set: Ideal exploration

Ideal exploration moves along the typical set



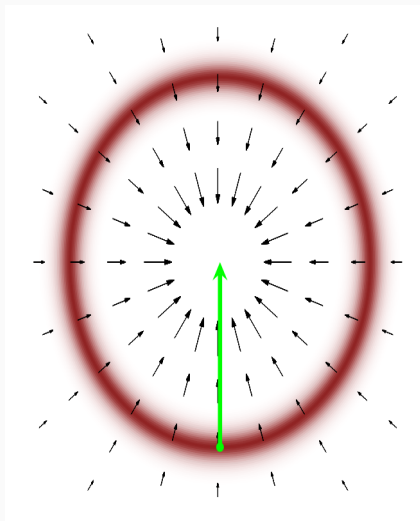
Vector fields

We should come up with a vector field aligned along the typical set



Vector fields and gradients

This vector field can be obtained indirectly from the gradient



Consider our K parameters θ_k as positions and lift them into a $2K$ dimensional position-momentum (phase) space

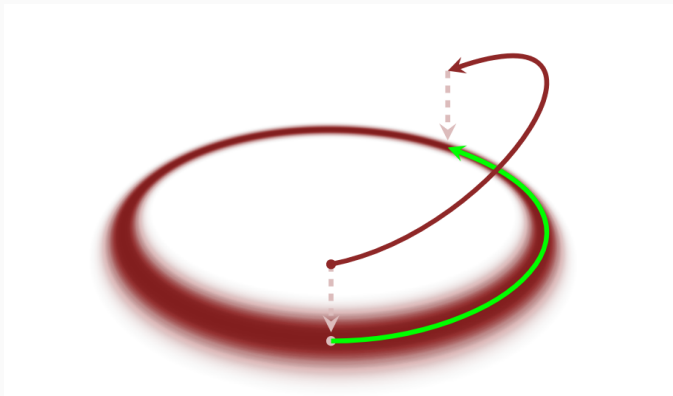
$$\theta_k \rightarrow (\theta_k, \phi_k)$$

with

$$p(\theta, \phi) = p(\phi \mid \theta)p(\theta)$$

Phase space exploration

Marginalizing out ϕ will give us back our target distribution on θ



Intuition about Hamiltonian Monte Carlo (HMC)

We aim to make the *Hamiltonian* function

$$H(\theta, \phi) := -\log p(\theta, \phi) = -\log p(\phi \mid \theta) - \log p(\theta)$$

energy conserving, where we interpret

- $K(\theta, \phi) = -\log p(\phi \mid \theta)$ as the *kinetic energy*
- $V(\theta) = -\log p(\theta)$ as the *potential energy*

Any trajectory in time t that follow Hamilton's equations

$$\begin{aligned}\frac{d\theta}{dt} &= +\frac{\partial H}{\partial \phi} = \frac{\partial K}{\partial \phi} \\ \frac{d\phi}{dt} &= -\frac{\partial H}{\partial \theta} = -\frac{\partial K}{\partial \theta} - \frac{\partial V}{\partial \theta}\end{aligned}$$

follows the vector field along the typical set of θ

HMC: interactive example

<https://chi-feng.github.io/mcmc-demo/app.html?algorithm=HamiltonianMC&target=banana>

Probabilistic Programming: Stan



- Probabilistic programming language written in C++ to fit open-ended Bayesian models
- Powerful sampling algorithms: adaptive HMC
- Automatic differentiation library
- Runs on all major operating systems
- Can be called from all major interpreted languages

Website: <https://mc-stan.org/>

Binomial Model in Stan

```
data {  
  int<lower=1> N;  
  int<lower=0,upper=N> y;  
}  
parameters {  
  real<lower=0,upper=1> theta;  
}  
model {  
  y ~ binomial(N, theta);  
  theta ~ beta(1, 1);  
}
```

Stan Syntax: Model Blocks

```
functions
  // user defined Stan functions
data
  // data passed by the user
transformed data
  // variables depending on the data block
  // computed only once before fitting the model
parameters
  // unknown variables to be sampled
transformed parameters
  // variables depending on data and parameter blocks
model
  // specification of the log-posterior density
  // defined variables are local
generated quantities
  // variables to be computed after the model fitting
  // not included in the actual sampling process
```

Example: Binomial Model in CmdStan(R)

Suppose we have observed $y = 4$ out of $N = 10$ people having the genetic variant of interest:

Load cmdstanr:

```
library(cmdstanr)
```

Compile the Stan model:

```
mod <- cmdstan_model(stan_file = "binomial_model.stan")
```

Fit the Stan model

```
fit_mcmc <- mod$sample(  
  data = stan_data,  
  chains = 4  
)
```

Example: Investigate the posterior draws

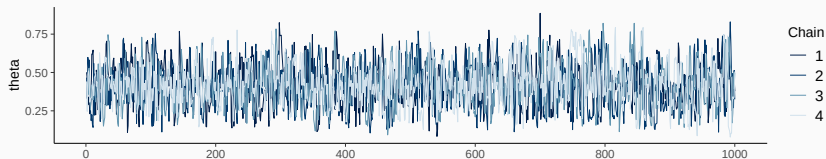
```
fit_mcmc$summary()
```

```
# A tibble: 2 x 10
```

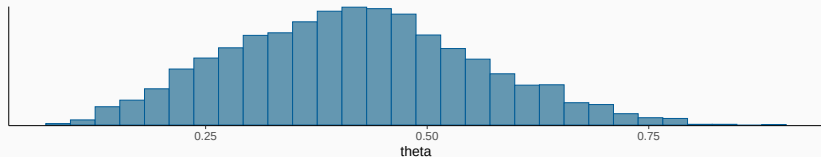
	variable	mean	median	sd	mad	q5	q95	rhat	ess_bulk	ess_tail
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	lp__	-8.67	-8.40	0.720	0.336	-10.2	-8.15	1.00	1611.	2117.
2	theta	0.419	0.412	0.138	0.146	0.205	0.656	1.00	1432.	1954.

Example: Visualize the Posterior

```
draws <- fit_mcmc$draws()
bayesplot::mcmc_trace(draws, pars = "theta")
```



```
bayesplot::mcmc_hist(draws, pars = "theta")
```



Variational inference

Key ideas:

- Approximate the posterior with a parametric distribution
- Turn posterior estimation into an optimization problem
- Usually faster than MCMC but less accurate

Definition for two continuous distributions with densities $p(x)$ and $q(x)$

$$KL(p(x) || q(x)) = \int \log \left(\frac{p(x)}{q(x)} \right) p(x) dx$$

- $p(x)$: the (target) distribution to be approximated
- $q(x)$: The distribution attempting to approximate $p(x)$

Variational Inference: Reverse KL loss

We use KL divergence in VI as follows:

$$\text{minimize } KL(p_A(\theta | y) || p(\theta | y)) = \int \log \left(\frac{p_A(\theta | y)}{p(\theta | y)} \right) p_A(\theta | y) d\theta$$

- $p(\theta | y)$: The target posterior
- $p_A(\theta | y)$: The variational approximation of $p(\theta | y)$

Notice that we apply the KL divergence in reverse order!

Simplification:

$$KL(p_A(\theta | y) || p(\theta | y)) \propto \int \log \left(\frac{p_A(\theta | y)}{p(y | \theta) p(\theta)} \right) p_A(\theta | y) d\theta$$

Do you see why this simplification is important?

Minimizing the KL loss in practice

The KL integral is usually not analytically tractable

We approximate it via sampling:

$$\text{minimize} \quad \frac{1}{S} \sum_{s=1}^S \log \left(\frac{p_A(\theta^{(s)} | y)}{p(y | \theta^{(s)}) p(\theta^{(s)})} \right) \quad \text{for} \quad \theta^{(s)} \sim p_A(\theta | y)$$

Writing it in terms of log densities:

$$\text{minimize} \quad \frac{1}{S} \sum_{s=1}^S \log p_A(\theta^{(s)} | y) - \log p(y | \theta^{(s)}) - \log p(\theta^{(s)})$$

We optimize this loss using stochastic gradient descent (SGD)

Variational Inference: Normal approximation

In most cases, we use a multivariate normal distribution as variational approximation:

$$p_A(\theta \mid y) = \text{MultivariateNormal}(\mu_A, \Sigma_A)$$

- μ_A is the mean vector of K
- Σ_A is the covariance matrix of shape $K \times K$
- K is the number of model parameters θ

Variational parameters to be estimated are μ_A and Σ_A

Other variational approximations that normal are of course possible

Variational Inference: Fullrank vs. Meanfield

Fullrank VI:

- Optimize μ_A and the whole Σ_A
- Captures the correlation structure of θ
- Requires the estimation of $O(K^2)$ variational parameters

Meanfield VI:

- Fix the off-diagonal elements of Σ_A to 0
- Optimize μ_A and the diagonal elements of Σ_A
- Does not captures the correlation structure of θ
- Requires the estimation of $O(K)$ variational parameters

Stan supports both fullrank and meanfield VI:

```
fit_vi <- mod$variational(  
  data = stan_data,  
  algorithm = "meanfield"  
)
```

```
fit_vi$summary()
```

```
# A tibble: 3 x 7
```

	variable	mean	median	sd	mad	q5	q95
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	lp__	-3.38	-3.05	0.799	0.344	-4.99	-2.81
2	lp_approx__	-0.479	-0.206	0.699	0.286	-1.81	-0.00158
3	theta	0.431	0.428	0.143	0.149	0.197	0.670

Importance Sampling (Bayesian Version)

Importance sampling with the prior as proposal

Suppose we use the prior $p(\theta)$ as proposal for the posterior $p(\theta | y)$.

We only have access to the unnormalized posterior density

$$\tilde{p}(\theta | y) = p(y | \theta) p(\theta) \propto p(\theta | y).$$

This results in the self-normalized importance sampling approximation:

$$\mathbb{E}_{\theta|y}(f(\theta)) = \frac{\int f(\theta) r(\theta) p(\theta) d\theta}{\int r(\theta) p(\theta) d\theta} = \frac{\int f(\theta) p(y | \theta) p(\theta) d\theta}{\int p(y | \theta) p(\theta) d\theta}$$

since the importance ratios evaluate to:

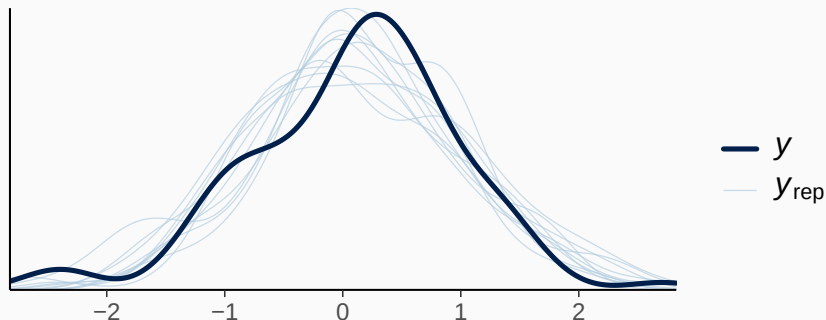
$$r(\theta) = \frac{p(y | \theta) p(\theta)}{p(\theta)} = p(y | \theta)$$

Importance sampling with the prior as proposal is as bad as Monte Carlo estimation of the marginal likelihood (very bad).

The posterior predictive distribution

The posterior predictive distribution is the distribution of data as implied by the model's posterior:

$$p(y_{\text{rep}} | y) = \int p(y_{\text{rep}} | \theta) p(\theta | y) d\theta$$



Bayesian Leave-One-Out Cross-Validation

Predict a single observation y_i by all other observations y_{-i} :

$$p(y_i | y_{-i}) = \int p(y_i | \theta) p(\theta | y_{-i}) d\theta$$

Summarize via the ELPD measure:

$$\text{ELPD}_{\text{LOO}} = \sum_{i=1}^N \log p(y_i | y_{-i}) \quad \text{LOOIC} = -2 \text{ELPD}_{\text{LOO}}$$

(Other measures are possible as well)

Important properties of LOO-CV:

- All possible N splits can be evaluated
- Can be approximated using the full model

Importance sampling for Bayesian LOO-CV

Idea: Use the full posterior $p(\theta \mid y)$ as proposal for the targets $p(\theta \mid y_{-i})$.

Compute the raw LOO importance ratios for a factorized likelihood:

$$r_i^{(s)} = \frac{p(\theta^{(s)} \mid y_{-i})}{p(\theta^{(s)} \mid y)} \propto \frac{\prod_{j=1, \dots, i-1, i+1, \dots, N} p(y_j \mid \theta^{(s)}) p(\theta^{(s)})}{\prod_{j=1, \dots, N} p(y_j \mid \theta^{(s)}) p(\theta^{(s)})} = \frac{1}{p(y_i \mid \theta^{(s)})}$$

Approximate the i th posterior predictive density:

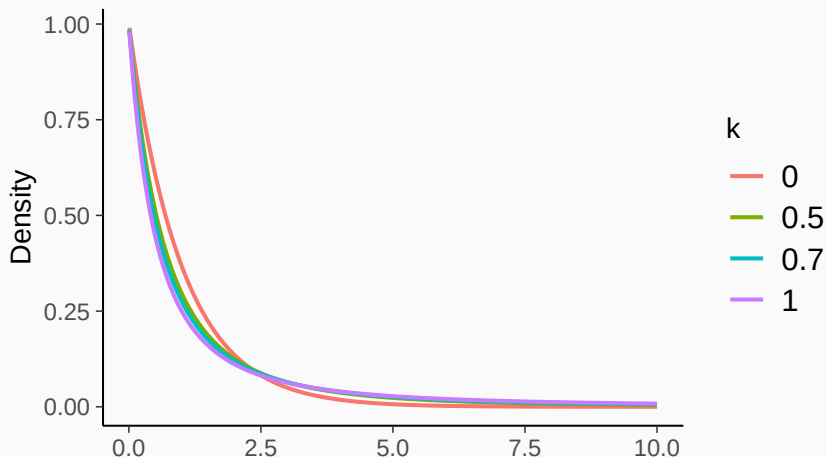
$$p(y_i \mid y_{-i}) \approx \frac{\sum_{s=1}^S p(y_i \mid \theta^{(s)}) r_i^{(s)}}{\sum_{s=1}^S r_i^{(s)}}$$

Sum over the log pointwise contributions:

$$\text{ELPD}_{\text{LOO}} = \sum_{i=1}^N \log p(y_i \mid y_{-i})$$

Pareto Smoothed Importance Sampling (PSIS)

Replace the largest importance ratios with quantiles of the generalized Pareto distribution (GPD)



The $\hat{k}$ -Diagnostic

The number of existing moments of the GPD is

$$\# \text{moments} = \begin{cases} \text{if } k > 0 : \text{floor}\left(\frac{1}{k}\right) \\ \text{else: } \infty \end{cases}$$

Relevant thresholds:

- $k < 0.5$: Finite variance and fast convergence rate
- $0.5 \leq k \leq 0.7$: Convergence rate is still okay
- $k > 0.7$: Pre-asymptotic behavior gets in your way
- $k > 1$: All is lost